

Complete RAG Stage-1 Competition Roadmap

From 0.511 Combined Score to 0.85+ Top-3 Qualification

CURRENT SYSTEM CRITICAL ANALYSIS

Your Performance Reality Check:

- **Recall@5: 0.200** (Missing 80% of relevant documents) - 50% of score
- **Recall@10: 0.200** (No improvement with more results - major red flag)
- **Precision@5: 0.200** (Only 1 in 5 documents relevant) - 20% of score
- **Precision@10: 0.100** (Only 1 in 10 documents relevant - critical failure)
- **NDCG@5: 1.000** (Perfect ranking when documents found) - 30% of score
- **Combined Score: 0.511** (Bottom 50% - elimination territory)

Root Cause Analysis:

1. **Retrieval Strategy Failure:** Single embedding model with basic similarity search cannot handle diverse query types
2. **No Query Understanding:** Missing query expansion, entity recognition, multi-hop detection
3. **Insufficient Coverage:** Not casting wide enough net to find relevant documents
4. **Poor Precision Filtering:** No relevance scoring or document quality assessment
5. **Single Domain Approach:** Same strategy for scientific papers, legal docs, and QA datasets

90-DAY TRANSFORMATION ROADMAP

PHASE 1: FOUNDATION REBUILD (Days 1-21)

Target: Achieve 0.60+ Recall, 0.40+ Precision

Week 1 (Days 1-7): Emergency Architecture Overhaul

Day 1-2: Multi-Retrieval Infrastructure

```
python
```

Replace your single model approach with ensemble architecture

class CompetitionRAGSystem:

def __init__(self):

Stage 1: Multiple retrieval strategies

self.retrievers = {

'bm25': BM25PlusRetriever(), *# Keyword matching*

'mpnet': DenseRetriever('all-mpnet-base-v2'), *# General embeddings*

'minilm': DenseRetriever('all-MiniLM-L12-v2'), *# Fast embeddings*

'e5': DenseRetriever('intfloat/e5-large-v2'), *# Query-optimized*

'bge': DenseRetriever('BAAI/bge-large-en-v1.5') *# Chinese-English*

}

Stage 2: Domain-specific models

self.domain_retrievers = {

'scientific': DenseRetriever('allenai/specter'),

'legal': DenseRetriever('nlpaueb/legal-bert-base-uncased'),

'medical': DenseRetriever('emilyalsentzer/Bio_ClinicalBERT')

}

Stage 3: Reranking pipeline

self.cross_encoder = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-12-v2')

Critical Change 1: Retrieval Scale

- **Current:** Retrieve k=5 or k=10 documents
- **New:** Retrieve k=1000+ candidates, then filter down
- **Reason:** Your 20% recall means relevant documents exist but aren't being found

Critical Change 2: Query Processing

python

```

def process_query(self, query: str) -> List[str]:
    """Generate 5-7 query variations for comprehensive coverage"""
    expansions = [query] # Original

    # Paraphrasing
    expansions.append(self.paraphrase_query(query))

    # Entity extraction and expansion
    entities = self.extract_entities(query)
    for entity in entities[:2]:
        expansions.append(f"{query} {entity}")

    # Question type transformation
    if "what is" in query.lower():
        expansions.extend([
            query.replace("what is", "definition of"),
            query.replace("what is", "explain"),
            query.replace("what is", "describe")
        ])

    # Keyword extraction
    keywords = self.extract_keywords(query)
    expansions.append(" ".join(keywords))

    return expansions[:7] # Limit to prevent noise

```

Day 3-4: Dataset-Specific Optimization

python

Address the 7 different dataset types with specialized strategies

```
DATASET_CONFIGS = {  
    'cord19': { # COVID research papers  
        'primary_model': 'allenai/specter',  
        'chunk_size': 1500,  
        'query_boost': ['covid', 'coronavirus', 'pandemic', 'disease'],  
        'rerank_threshold': 0.3  
    },  
    'wikihop': { # Multi-hop reasoning  
        'primary_model': 'sentence-transformers/multi-qa-mpnet-base-dot-v1',  
        'chunk_size': 800,  
        'multi_hop_detection': True,  
        'entity_linking': True  
    },  
    'hotpotqa': { # Complex QA  
        'primary_model': 'all-mpnet-base-v2',  
        'chunk_size': 1000,  
        'question_decomposition': True,  
        'answer_type_prediction': True  
    },  
    'eu_legislation': { # Legal documents  
        'primary_model': 'nlpauieb/legal-bert-base-uncased',  
        'chunk_size': 2000,  
        'legal_entity_recognition': True,  
        'section_awareness': True  
    }  
}
```

Day 5-7: Evaluation Framework Implementation

python

```
class CompetitionEvaluator:
    def evaluate_system(self, queries, ground_truth, k_values=[5, 10]):
        results = {}

        for k in k_values:
            # Recall@k (50% weight)
            recall = self.calculate_recall_at_k(queries, ground_truth, k)

            # Precision@k (20% weight)
            precision = self.calculate_precision_at_k(queries, ground_truth, k)

            # NDCG@k (30% weight)
            ndcg = self.calculate_ndcg_at_k(queries, ground_truth, k)

            # Competition score
            combined = 0.5 * recall + 0.2 * precision + 0.3 * ndcg

            results[f'k_{k}'] = {
                'recall': recall,
                'precision': precision,
                'ndcg': ndcg,
                'combined': combined
            }

        return results
```

Week 2 (Days 8-14): Advanced Retrieval Pipeline

Day 8-10: Multi-Stage Retrieval

python

```

def retrieve_documents(self, query: str, k: int = 10) -> List[Dict]:
    """
    Three-stage retrieval for maximum recall + precision
    """
    # Stage 1: Wide net retrieval (maximize recall)
    candidates = {}
    query_variations = self.process_query(query)

    for q_var in query_variations:
        # BM25 sparse retrieval
        bm25_results = self.retrievers['bm25'].search(q_var, k=500)
        self._merge_candidates(candidates, bm25_results, 'bm25')

        # Dense retrieval ensemble
        for model_name, retriever in self.retrievers.items():
            if model_name != 'bm25':
                dense_results = retriever.search(q_var, k=300)
                self._merge_candidates(candidates, dense_results, model_name)

    # Stage 2: Score fusion and filtering
    fused_candidates = self._fuse_scores(candidates)[:200]

    # Stage 3: Cross-encoder reranking
    reranked = self.cross_encoder.rerank(query, fused_candidates, k)

    return reranked

```

Day 11-14: Chunking Strategy Optimization

python

```

class MultiChunkingStrategy:
    """
    Multiple chunking approaches for different document types
    """
    def __init__(self):
        self.strategies = {
            'sentence': SentenceTextSplitter(),
            'recursive_512': RecursiveCharacterTextSplitter(chunk_size=512, overlap=100),
            'recursive_1000': RecursiveCharacterTextSplitter(chunk_size=1000, overlap=200),
            'recursive_1500': RecursiveCharacterTextSplitter(chunk_size=1500, overlap=300),
            'paragraph': ParagraphTextSplitter()
        }

    def chunk_document(self, text: str, doc_type: str) -> List[Dict]:
        all_chunks = []

        # Use multiple chunking strategies
        for strategy_name, splitter in self.strategies.items():
            chunks = splitter.split_text(text)

            for i, chunk in enumerate(chunks):
                all_chunks.append({
                    'text': chunk,
                    'strategy': strategy_name,
                    'chunk_id': f"{strategy_name}_{i}",
                    'size': len(chunk)
                })

        return all_chunks

```

Week 3 (Days 15-21): Query Understanding & Expansion

Day 15-17: Advanced Query Processing

python

```

class QueryProcessor:
    def __init__(self):
        self.entity_extractor = EntityExtractor()
        self.query_classifier = QueryTypeClassifier()
        self.paraphraser = QueryParaphraser()

    def process_complex_query(self, query: str) -> Dict:
        """
        Comprehensive query understanding for competition datasets
        """
        analysis = {
            'original': query,
            'entities': self.entity_extractor.extract(query),
            'query_type': self.query_classifier.classify(query),
            'complexity': self.assess_complexity(query),
            'expansions': [],
            'keywords': self.extract_keywords(query)
        }

        # Multi-hop detection for WikiHop/HotpotQA
        if self.is_multi_hop(query):
            analysis['sub_queries'] = self.decompose_query(query)
            analysis['reasoning_type'] = 'multi_hop'

        # Generate expansions based on query type
        analysis['expansions'] = self.generate_targeted_expansions(analysis)

        return analysis

    def generate_targeted_expansions(self, analysis: Dict) -> List[str]:
        expansions = [analysis['original']]

        # Type-specific expansions
        if analysis['query_type'] == 'definition':
            expansions.extend([
                analysis['original'].replace('what is', 'definition'),
                analysis['original'].replace('what is', 'meaning'),
                analysis['original'].replace('what is', 'explanation')
            ])

        elif analysis['query_type'] == 'procedural':
            expansions.extend([
                analysis['original'].replace('how to', 'method'),
                analysis['original'].replace('how to', 'process'),
                analysis['original'].replace('how to', 'procedure')
            ])

```



```
# Entity-based expansions
```

```
for entity in analysis['entities']:
```

```
    expansions.append(f"{analysis['original']} {entity}")
```

```
    expansions.append(entity) # Entity-only search
```

```
# Keyword-only search
```

```
expansions.append(' '.join(analysis['keywords']))
```

```
return list(set(expansions))[:10] # Dedupe and limit
```

Day 18-21: Domain Adaptation

```
python
```

```

class DomainAdaptiveRetriever:
    def __init__(self):
        self.domain_classifier = DomainClassifier()
        self.domain_configs = DATASET_CONFIGS

    def adapt_retrieval_strategy(self, query: str, documents: List[Dict]) -> str:
        """
        Select optimal retrieval strategy based on domain
        """
        # Classify query domain
        domain = self.domain_classifier.predict(query)

        # Classify document domains
        doc_domains = [self.classify_document(doc) for doc in documents[:100]]
        dominant_domain = max(set(doc_domains), key=doc_domains.count)

        # Select strategy
        if domain in self.domain_configs:
            return domain
        elif dominant_domain in self.domain_configs:
            return dominant_domain
        else:
            return 'general'

    def retrieve_with_domain_adaptation(self, query: str, k: int = 10):
        domain_strategy = self.adapt_retrieval_strategy(query, self.documents)
        config = self.domain_configs[domain_strategy]

        # Use domain-specific model
        primary_model = self.domain_models[config['primary_model']]

        # Domain-specific query processing
        if config.get('multi_hop_detection'):
            query_analysis = self.detect_multi_hop_reasoning(query)
        else:
            query_analysis = self.standard_query_processing(query)

        # Retrieve with domain optimization
        results = primary_model.search(query, k=k*5)

        # Domain-specific post-processing
        if config.get('legal_entity_recognition'):
            results = self.boost_legal_entities(results, query)

        return results

```

PHASE 2: PRECISION OPTIMIZATION (Days 22-42)

Target: Achieve 0.80+ Recall, 0.65+ Precision

Week 4 (Days 22-28): Cross-Encoder Reranking

Day 22-24: Advanced Reranking Pipeline

```
python
```

```

class MultiStageReranker:
    def __init__(self):
        self.cross_encoders = {
            'general': CrossEncoder('cross-encoder/ms-marco-MiniLM-L-12-v2'),
            'scientific': CrossEncoder('cross-encoder/ms-marco-electra-base'),
            'conversational': CrossEncoder('cross-encoder/qnli-electra-base')
        }

        self.relevance_classifier = RelevanceClassifier()

    def rerank_candidates(self, query: str, candidates: List[Dict], k: int = 10):
        """
        Multi-stage reranking for precision optimization
        """

        if len(candidates) == 0:
            return []

        # Stage 1: Relevance filtering
        relevant_candidates = []
        for candidate in candidates:
            relevance_score = self.relevance_classifier.score(query, candidate['text'])
            if relevance_score > 0.3: # Threshold tuned on validation data
                candidate['relevance_score'] = relevance_score
                relevant_candidates.append(candidate)

        # Stage 2: Cross-encoder scoring
        query_doc_pairs = [(query, candidate['text'][:512]) for candidate in relevant_candidates]
        ce_scores = self.cross_encoders['general'].predict(query_doc_pairs)

        # Stage 3: Combined scoring
        for i, candidate in enumerate(relevant_candidates):
            candidate['ce_score'] = float(ce_scores[i])
            candidate['final_score'] = (
                0.4 * candidate['relevance_score'] +
                0.6 * candidate['ce_score']
            )

        # Final ranking
        reranked = sorted(relevant_candidates, key=lambda x: x['final_score'], reverse=True)
        return reranked[:k]

```

Day 25-28: Precision Filtering

python

```

class PrecisionOptimizer:
    def __init__(self):
        self.semantic_scorer = SemanticSimilarityScorer()
        self.keyword_scorer = KeywordOverlapScorer()
        self.context_scorer = ContextualRelevanceScorer()

    def filter_for_precision(self, query: str, candidates: List[Dict], threshold: float = 0.5):
        """
        Multi-dimensional relevance filtering
        """
        filtered_candidates = []

        for candidate in candidates:
            doc_text = candidate['text']

            # Multiple relevance signals
            semantic_score = self.semantic_scorer.score(query, doc_text)
            keyword_score = self.keyword_scorer.score(query, doc_text)
            context_score = self.context_scorer.score(query, doc_text)

            # Weighted combination
            combined_relevance = (
                0.5 * semantic_score +
                0.3 * keyword_score +
                0.2 * context_score
            )

            if combined_relevance >= threshold:
                candidate['precision_score'] = combined_relevance
                filtered_candidates.append(candidate)

        return sorted(filtered_candidates, key=lambda x: x['precision_score'], reverse=True)

```

Week 5 (Days 29-35): Ensemble Optimization

Day 29-31: Score Fusion Optimization

python

```

class OptimalScoreFusion:
    def __init__(self):
        # Weights optimized through Bayesian optimization
        self.fusion_weights = {
            'bm25': 0.25,
            'mpnet': 0.20,
            'minilm': 0.15,
            'e5': 0.15,
            'bge': 0.10,
            'domain_specific': 0.15
        }

        self.normalization_params = {
            'bm25': {'min': 0, 'max': 50},
            'dense': {'min': -1, 'max': 1}
        }

    def optimize_fusion_weights(self, validation_queries, ground_truth):
        """
        Bayesian optimization of fusion weights
        """
        import optuna

        def objective(trial):
            # Suggest weights
            weights = {
                'bm25': trial.suggest_float('bm25', 0.1, 0.5),
                'mpnet': trial.suggest_float('mpnet', 0.1, 0.3),
                'minilm': trial.suggest_float('minilm', 0.1, 0.3),
                'e5': trial.suggest_float('e5', 0.1, 0.3),
                'bge': trial.suggest_float('bge', 0.05, 0.2),
                'domain_specific': trial.suggest_float('domain', 0.1, 0.3)
            }

            # Normalize weights
            total = sum(weights.values())
            weights = {k: v/total for k, v in weights.items()}

            # Evaluate with these weights
            self.fusion_weights = weights
            score = self.evaluate_on_validation(validation_queries, ground_truth)

            return score['combined']

        study = optuna.create_study(direction='maximize')
        study.optimize(objective, n_trials=100)

```

```
self.fusion_weights = study.best_params
return self.fusion_weights
```

Day 32-35: Ensemble Method Implementation

python

```
class CompetitionEnsemble:
    def __init__(self):
        self.base_retrievers = [
            BM25Retriever(),
            DenseRetriever('all-mpnet-base-v2'),
            DenseRetriever('all-MiniLM-L12-v2'),
            DenseRetriever('intfloat/e5-large-v2'),
            DenseRetriever('BAAI/bge-large-en-v1.5')
        ]

        self.meta_ranker = MetaRanker() # Learns to combine retriever outputs

    def ensemble_retrieve(self, query: str, k: int = 10):
        """
        Advanced ensemble retrieval with meta-learning
        """
        # Get predictions from all base retrievers
        retriever_results = {}
        for i, retriever in enumerate(self.base_retrievers):
            results = retriever.search(query, k=100)
            retriever_results[f'retriever_{i}'] = results

        # Meta-ranker combines results intelligently
        final_ranking = self.meta_ranker.rank(query, retriever_results, k)

        return final_ranking
```

Week 6 (Days 36-42): Hyperparameter Optimization

Day 36-42: Systematic Parameter Tuning

python

```

class HyperparameterOptimizer:
    def __init__(self):
        self.param_space = {
            'bm25_k1': (0.5, 3.0),
            'bm25_b': (0.3, 0.9),
            'chunk_size': (256, 2048),
            'chunk_overlap': (50, 500),
            'retrieval_k': (50, 1000),
            'rerank_threshold': (0.1, 0.7),
            'fusion_weights': {
                'bm25': (0.1, 0.5),
                'dense': (0.5, 0.9)
            }
        }

    def optimize_for_competition_metrics(self, train_queries, validation_queries):
        """
        Multi-objective optimization for Recall, Precision, and NDCG
        """
        best_params = {}
        best_score = 0

        # Grid search with random sampling
        for trial in range(200):
            params = self.sample_parameters()

            # Configure system with sampled parameters
            system = self.configure_system(params)

            # Evaluate on validation set
            metrics = system.evaluate(validation_queries)

            # Competition scoring
            competition_score = (
                0.5 * metrics['recall@10'] +
                0.2 * metrics['precision@10'] +
                0.3 * metrics['ndcg@10']
            )

            if competition_score > best_score:
                best_score = competition_score
                best_params = params.copy()

        return best_params, best_score

```


PHASE 3: COMPETITION OPTIMIZATION (Days 43-63)

Target: Achieve 0.90+ Recall, 0.75+ Precision, 0.85+ Combined Score

Week 7 (Days 43-49): Dataset-Specific Fine-tuning

Day 43-45: Per-Dataset Optimization

python

```
class DatasetSpecificOptimizer:
    def __init__(self):
        self.dataset_optimizers = {
            'cord19': COVID19Optimizer(),
            'wikihop': MultiHopOptimizer(),
            'hotpotqa': ComplexQAOptimizer(),
            'eu_legislation': LegalDocOptimizer(),
            'apt_notes': SecurityOptimizer(),
            'robustqa': RobustQAOptimizer(),
            'multihop_qa': MultiHopQAOptimizer()
        }

    def optimize_for_dataset(self, dataset_name: str, queries: List, documents: List):
        """
        Apply dataset-specific optimizations
        """
        optimizer = self.dataset_optimizers[dataset_name]

        # Dataset-specific preprocessing
        processed_docs = optimizer.preprocess_documents(documents)
        processed_queries = optimizer.preprocess_queries(queries)

        # Dataset-specific retrieval strategy
        retrieval_config = optimizer.get_optimal_config()

        # Train dataset-specific components
        if hasattr(optimizer, 'fine_tune'):
            optimizer.fine_tune(processed_queries, processed_docs)

        return retrieval_config
```

Day 46-49: Multi-hop Reasoning Optimization

python

```
class MultiHopReasoningSystem:
```

```
    """
```

```
    Specialized system for WikiHop and HotpotQA multi-hop queries
```

```
    """
```

```
    def __init__(self):
```

```
        self.query_decomposer = QueryDecomposer()
```

```
        self.entity_linker = EntityLinker()
```

```
        self.reasoning_tracker = ReasoningChainTracker()
```

```
    def process_multihop_query(self, query: str) -> Dict:
```

```
        """
```

```
        Decompose multi-hop queries into sub-queries
```

```
        """
```

```
        # Detect if query requires multi-hop reasoning
```

```
        is_multihop = self.detect_multihop(query)
```

```
        if not is_multihop:
```

```
            return {'sub_queries': [query], 'reasoning_type': 'single_hop'}
```

```
        # Decompose into sub-queries
```

```
        sub_queries = self.query_decomposer.decompose(query)
```

```
        # Extract entities for linking
```

```
        entities = self.entity_linker.extract_and_link(query)
```

```
        # Build reasoning chain
```

```
        reasoning_chain = self.reasoning_tracker.build_chain(sub_queries, entities)
```

```
        return {
```

```
            'sub_queries': sub_queries,
```

```
            'entities': entities,
```

```
            'reasoning_chain': reasoning_chain,
```

```
            'reasoning_type': 'multi_hop'
```

```
        }
```

```
    def retrieve_multihop(self, query: str, k: int = 10):
```

```
        """
```

```
        Retrieve documents for multi-hop reasoning
```

```
        """
```

```
        query_analysis = self.process_multihop_query(query)
```

```
        if query_analysis['reasoning_type'] == 'single_hop':
```

```
            return self.standard_retrieve(query, k)
```

```
        # Multi-hop retrieval
```

```
        all_candidates = {}
```

```
# Retrieve for each sub-query
for sub_query in query_analysis['sub_queries']:
    sub_results = self.standard_retrieve(sub_query, k=50)
    self.merge_results(all_candidates, sub_results)

# Entity-based retrieval
for entity in query_analysis['entities']:
    entity_results = self.entity_retrieve(entity, k=30)
    self.merge_results(all_candidates, entity_results)

# Reasoning chain filtering
filtered_results = self.reasoning_tracker.filter_by_chain(
    all_candidates, query_analysis['reasoning_chain']
)

return filtered_results[:k]
```

Week 8 (Days 50-56): Advanced Techniques Implementation

Day 50-52: ColBERT-style Late Interaction

python

```
class LateInteractionRetriever:
```

```
    """
```

```
ColBERT-inspired late interaction for better semantic matching
```

```
    """
```

```
def __init__(self):
```

```
    self.token_encoder = TokenEncoder()
```

```
    self.interaction_scorer = InteractionScorer()
```

```
def encode_tokens(self, text: str):
```

```
    """
```

```
Encode each token separately for late interaction
```

```
    """
```

```
tokens = self.tokenize(text)
```

```
token_embeddings = self.token_encoder.encode(tokens)
```

```
return token_embeddings
```

```
def late_interaction_score(self, query_tokens, doc_tokens):
```

```
    """
```

```
Compute late interaction similarity score
```

```
    """
```

```
# Max-sim operation between query and document tokens
```

```
scores = []
```

```
for q_token in query_tokens:
```

```
    max_sim = max([self.cosine_sim(q_token, d_token) for d_token in doc_tokens])
```

```
    scores.append(max_sim)
```

```
return sum(scores) / len(scores)
```

```
def retrieve_with_late_interaction(self, query: str, k: int = 10):
```

```
    """
```

```
Retrieve using late interaction scoring
```

```
    """
```

```
query_tokens = self.encode_tokens(query)
```

```
candidates = []
```

```
for doc in self.documents:
```

```
    doc_tokens = self.encode_tokens(doc['text'][:512])
```

```
    score = self.late_interaction_score(query_tokens, doc_tokens)
```

```
    candidates.append({'document': doc, 'score': score})
```

```
# Sort and return top k
```

```
candidates.sort(key=lambda x: x['score'], reverse=True)
```

```
return candidates[:k]
```

python

```

class LearnedSparseRetriever:
    """
    SPLADE-inspired learned sparse retrieval
    """

    def __init__(self):
        self.sparse_encoder = SparseEncoder()
        self.vocabulary = self.load_vocabulary()

    def encode_sparse(self, text: str):
        """
        Encode text into sparse representation
        """

        # Get token importance scores
        token_scores = self.sparse_encoder.encode(text)

        # Create sparse vector
        sparse_vector = {}
        for token, score in token_scores.items():
            if score > 0.1: # Threshold for sparsity
                sparse_vector[token] = score

        return sparse_vector

    def sparse_similarity(self, query_sparse, doc_sparse):
        """
        Compute similarity between sparse representations
        """

        intersection = set(query_sparse.keys()) & set(doc_sparse.keys())

        if not intersection:
            return 0.0

        score = 0.0
        for token in intersection:
            score += query_sparse[token] * doc_sparse[token]

        return score

    def retrieve_sparse(self, query: str, k: int = 10):
        """
        Retrieve using learned sparse representations
        """

        query_sparse = self.encode_sparse(query)

        candidates = []
        for doc in self.documents:

```

```
doc_sparse = self.encode_sparse(doc['text'])
score = self.sparse_similarity(query_sparse, doc_sparse)
candidates.append({'document': doc, 'score': score})

candidates.sort(key=lambda x: x['score'], reverse=True)
return candidates[:k]
```

Week 9 (Days 57-63): Final System Integration

Day 57-59: Complete Pipeline Integration

python

```
class CompetitionRAGSystemFinal:
```

```
    """
```

```
    Final integrated system combining all optimizations
```

```
    """
```

```
    def __init__(self):
```

```
        # Core retrieval components
```

```
        self.multi_retriever = MultiRetrieverEnsemble()
```

```
        self.query_processor = AdvancedQueryProcessor()
```

```
        self.reranker = MultiStageReranker()
```

```
        self.precision_optimizer = PrecisionOptimizer()
```

```
        # Advanced components
```

```
        self.multihop_system = MultiHopReasoningSystem()
```

```
        self.late_interaction = LateInteractionRetriever()
```

```
        self.sparse_retriever = LearnedSparseRetriever()
```

```
        # Dataset-specific optimizers
```

```
        self.dataset_optimizers = DatasetSpecificOptimizerSuite()
```

```
        # Evaluation framework
```

```
        self.evaluator = CompetitionEvaluator()
```

```
    def retrieve_competition_grade(self, query: str, dataset_type: str = 'general', k: int = 10):
```

```
        """
```

```
        Competition-grade retrieval pipeline
```

```
        """
```

```
        # Stage 1: Query analysis and processing
```

```
        query_analysis = self.query_processor.analyze(query)
```

```
        # Stage 2: Dataset-specific optimization
```

```
        retrieval_strategy = self.dataset_optimizers.get_strategy(dataset_type)
```

```
        # Stage 3: Multi-method retrieval
```

```
        candidates = {}
```

```
        # Standard ensemble retrieval
```

```
        ensemble_results = self.multi_retriever.retrieve(query_analysis, k=500)
```

```
        self.merge_candidates(candidates, ensemble_results, 'ensemble')
```

```
        # Multi-hop retrieval (if applicable)
```

```
        if query_analysis['requires_multihop']:
```

```
            multihop_results = self.multihop_system.retrieve_multihop(query, k=200)
```

```
            self.merge_candidates(candidates, multihop_results, 'multihop')
```